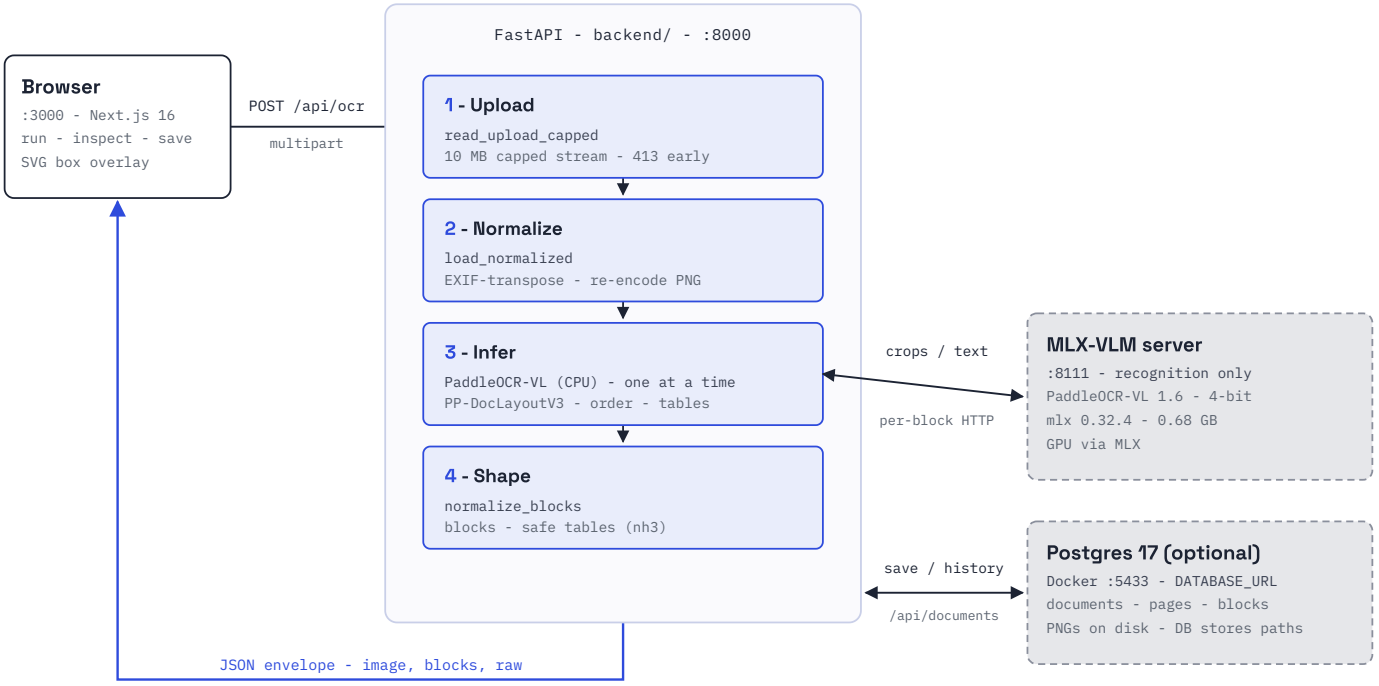
10

Invoice OCR Viewer

how it works - layout on the CPU, recognition on the GPU, everything on this Mac



One request, end to end. The backend does everything except recognition, which it hands to the MLX server - and nothing is stored unless you press Save.

blue = stages in the backend - blue line = the response - dashed = its own process (Postgres optional) - white = the browser

01 How a request flows

The backend handles everything between upload and JSON. Inference runs one page at a time behind a lock - no queue service.

1 - Upload

`read_upload_capped`

Streamed in 1 MB chunks and cut off at 10 MB, so a file that is too big fails fast - before it is ever held in memory.

2 - Normalize

`load_normalized`

Read by content, never by file extension. EXIF-rotated, re-encoded as PNG - and those exact bytes are what inference sees.

3 - Infer

`RealPipeline`

Check the MLX server is up, then run the pipeline: layout on the CPU, recognition sent block by block to the GPU. The model loads once and is reused; temp files are always cleaned up.

4 - Shape

`normalize_blocks`

Raw pipeline output becomes plain JSON: blocks in reading order, table HTML sanitized against an allowlist, temp-file paths stripped.

02 Why the boxes always line up

The boxes line up because there is nothing to convert:

- The backend normalizes the image first and hands those exact PNG bytes to the pipeline.
- The same bytes come back to your browser as the displayed image.
- So every `block_bbox` is already in image pixels - no conversion, no guessing.
- One SVG with `viewBox="0 0 width height"` sits on top of the `img` and scales with it, so boxes stay put at any size.
- Rotation and unwarping are switched off, so nothing invisible sits between a box and its pixels.

03 Trust boundaries

OCR output is handled like any untrusted input:

- Uploads are validated by decoding them and capped while streaming; CORS allows only localhost.
- Block text is rendered as text, never as HTML.
- Table HTML passes an nh3 allowlist server-side before anything is rendered.
- Selection is keyed on stable `block_id`s, never on text - documents repeat text.
- A failed run clears the previous result - you never see stale output.

04 Storage and review - optional

Persistence is opt-in. A run stores nothing by itself - you inspect the result first, then decide to save it.

- **Save to history** - one transaction writes the document, its pages and blocks; PNGs land in `data/documents/<id>/`, the DB keeps only their paths.
- **History page** (`/history`) - everything you saved, searchable, with review counts and delete. `/?doc=<id>` reopens a document in the viewer.
- **Corrections** - a fix is stored beside the original OCR text, never over it; later reads show the fix, the original stays.

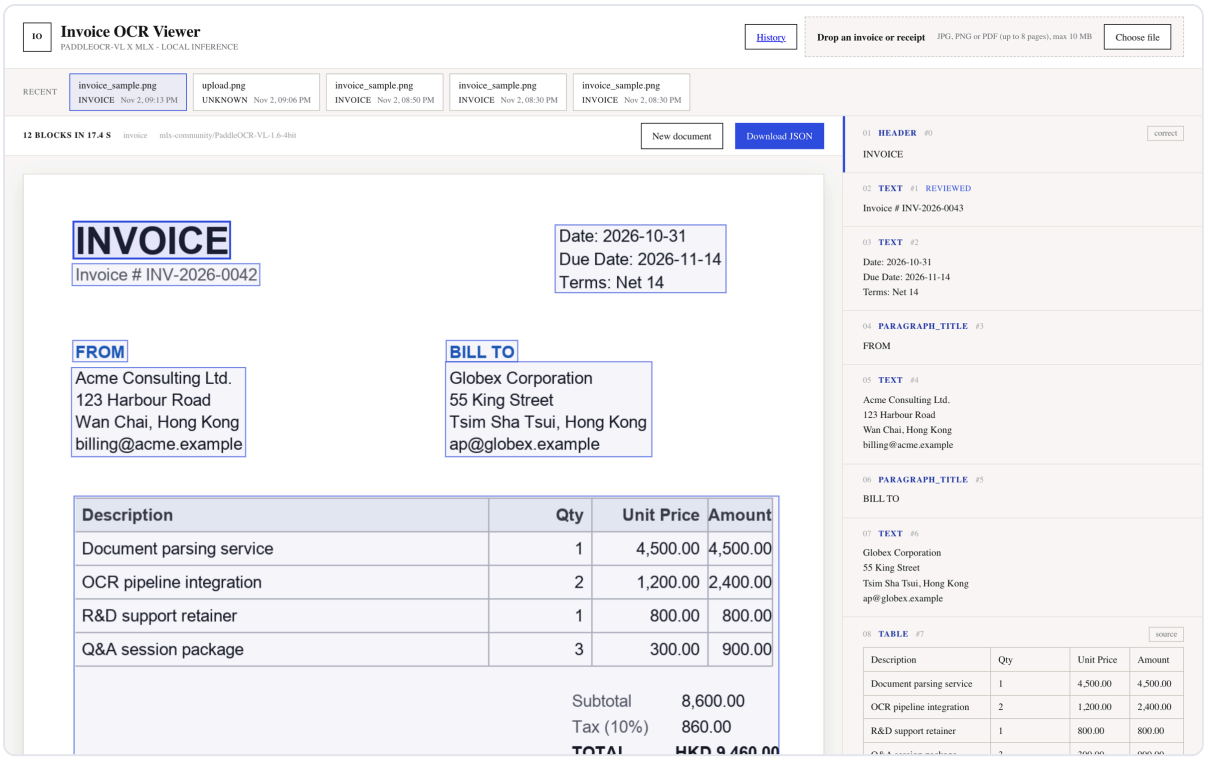
05 Stack and measured numbers

All numbers measured on the demo machine (Apple M2, 16 GB) - no claims beyond them.

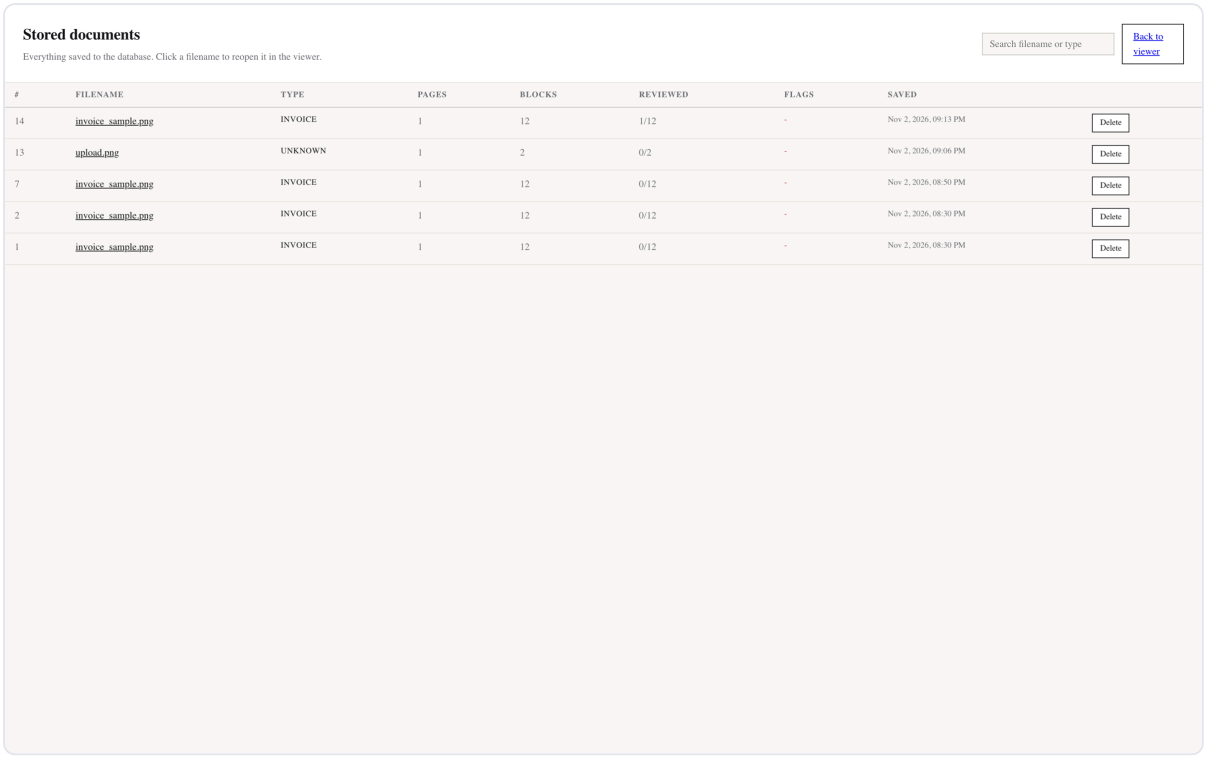
LAYER	TECHNOLOGY	ROLE
Viewer	`Next.js 16.4.3 - React 19.3`	Upload, run, inspect, correct, save, download - one screen
API	`FastAPI 0.162 - Python 3.11 (uv)`	Validates, normalizes, orchestrates; one envelope for every response
Layout	`paddleocr 3.7.0 - paddlepaddle 3.2.1 (CPU)`	PP-DocLayoutV3: regions, reading order, table structure
Recognition	`mlx-vlm 0.7.11 - mlx 0.32.4`	Serves `PaddleOCR-VL-1.6-4bit` (0.68 GB) on the GPU, recognition only
Storage	`Postgres 17 (Docker) - psycopg pool`	Optional: saved runs and corrections; PNGs on disk, paths in the DB
Latency	`12.6 - 21.4 s` per page	12 - 13 blocks on a typical invoice; the first request adds about 20 s one-time init
Tests	`74 + 8 + 15`	pytest without a model, 13 opt-in SQL tests, vitest, Playwright on a mocked API

06 What it looks like

Two screenshots from the running app - the real thing, not mockups.



The viewer with a saved invoice. Click a block on either side and both sides highlight it; the REVIEWED tag marks a block someone corrected earlier.



The history page - every saved document in one place, searchable. Click a filename to open it back in the viewer.

Everything runs locally - no cloud APIs, no accounts; Postgres is optional, for saved history. Generated from the invoice_ocr repo, 2026-11-02.